Section 15 : Inheritance

Information

- Title: `Inheritance`
- Authors: `Etienne KOA`

Aperçu

- Héritage
- Qu'est-ce que l'héritage ?
  - Pourquoi est-ce utile ?
- Terminologie et notation
- Héritage vs composition
- Dériver des classes à partir de classes existantes
  - Types d'héritage
- Membres protégés et accès aux cours
- Constructeurs et Destructeurs
  - Passer des arguments aux constructeurs de classes de base
  - Ordre des appels des constructeurs et des destructeurs
- Redéfinir les méthodes des classes de base
- Hiérarchies de classes
- Héritage multiple

Qu'est-ce que l'héritage ?

Qu'est-ce que c'est et pourquoi est-il utilisé ?

- Fournit une méthode pour créer de nouvelles classes à partir de classes existantes
- La nouvelle classe contient les données et les comportements de la classe existante
- Permettre la réutilisation des classes existantes
- Nous permet de nous concentrer sur les attributs communs à un ensemble de classes
- Permet aux nouvelles classes de modifier les comportements des classes existantes pour les rendre uniques
  - Sans réellement modifier la classe d'origine

Héritage

- Cours associés

  - `Player, Enemy, Level Boss, Hero, Super Player, etc.`
  - `Account, Savings Account, Checking Account, Trust Account, etc.`
  - `Shape, Line, Oval, Circle, Square, etc.`
  - `Person, Employee, Student, Faculty, Staff, Administrator, etc.`

Accounts

- Account
  - **balance, deposit, withdraw, ...**
- Savings Account
  - **balance, deposit, withdraw, interest, rate ...**
- Checking Account
  - **balance, deposit, withdraw, minimum balance, oer check free ...**
- Trust Account
  - **balance, deposit, withdraw, interest rate, ...**

Comptes - sans héritage - duplication de code

```
class Account {
    // balance, deposit, withdraw, . . .
};

class Savings_Account {
    // balance, deposit, withdraw, interest rate, . . .
};

class Checking_Account {
    // balance, deposit, withdraw, minimum balance, per check fee, . . .
};

class Trust_Account {
    // balance, deposit, withdraw, interest rate, . . .
};
```

Figure 1: Inheritance

Comptes - avec héritage - réutilisation du code

Terminologie et notation

Héritage - Terminologie

- Héritage
  - Processus de création de nouvelles classes à partir de classes existantes
  - Mécanisme de réutilisation
- Héritage unique
  - Une nouvelle classe est créée à partir d'une autre classe « unique »
- Héritage multiple

```
class Account {
    // balance, deposit, withdraw, . . .
};

class Savings_Account : public Account {
    // interest rate, specialized withdraw, . . .
};

class Checking_Account : public Account {
    // minimum balance, per check fee, specialized withdraw, . . .
};

class Trust_Account : public Account {
    // interest rate, specialized withdraw, . . .
};
```

Figure 2: Inheritance

  - Une nouvelle classe est créée à partir de deux (ou plusieurs) autres classes
- Classe de base (classe parent, super classe)
  - La classe étant étendue ou héritée de
- Classe dérivée (classe enfant, sous-classe)
  - La classe créée à partir de la classe Base
  - Héritera des attributs et des opérations de la classe Base
- Relation `Is-A`
  - Héritage public
  - Les classes dérivées sont des sous-types de leurs classes de base
  - Peut utiliser un objet de classe dérivé partout où nous utilisons un objet de classe de base
- Généralisation
  - Combiner des classes similaires en une seule classe plus générale basée sur des attributs communs
- Spécialisation
  - Création de nouvelles classes à partir d'attributs ou d'opérations existants prouvant des attributs plus spécialisés
- Héritage et hiérarchies de classes
  - Organisation de nos relations successorales

Hiérarchie des classes

- Cours :
  - A
  - B est dérivé de A
  - C est dérivé de A
  - D est dérivé de C
  - E est dérivé de D
- Cours :

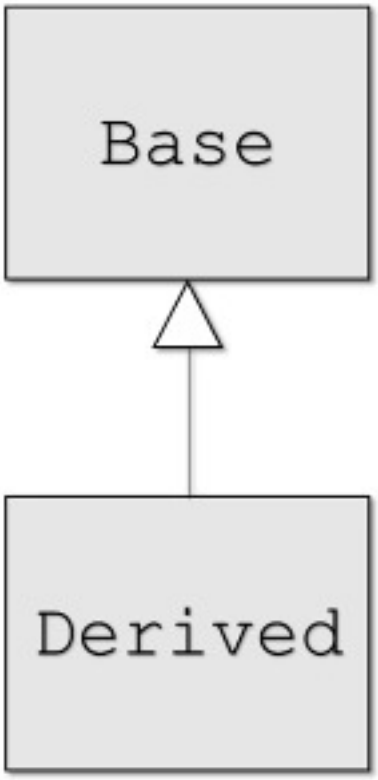

Figure 3: Inheritance

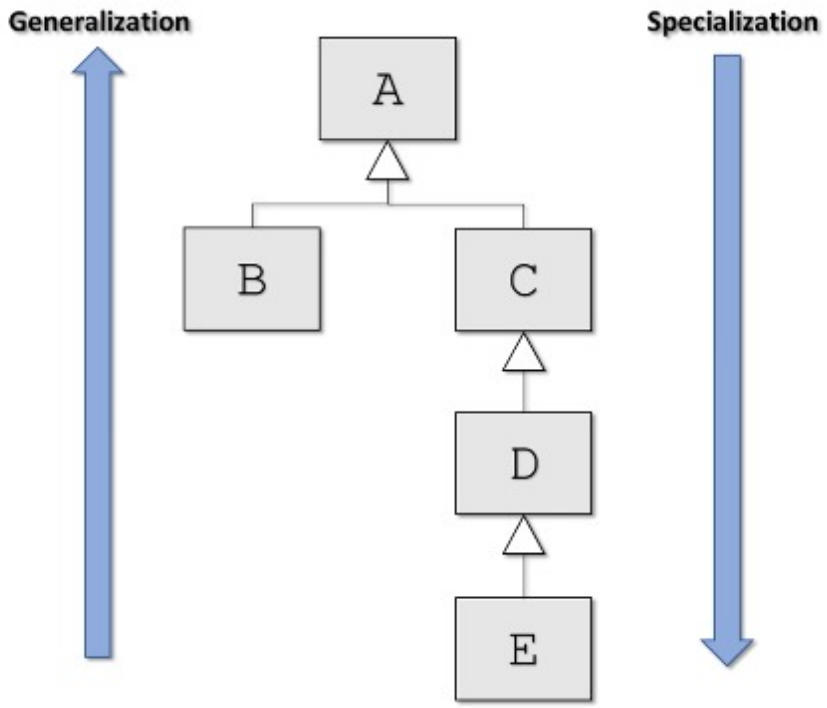

Figure 4: Inheritance

  - Personne
  - L'employé est dérivé de Personne
  - L'étudiant est dérivé de la personne
  - La faculté est dérivée de l'employé
  - Le personnel est dérivé de l'employé
  - L'administrateur est dérivé de l'employé

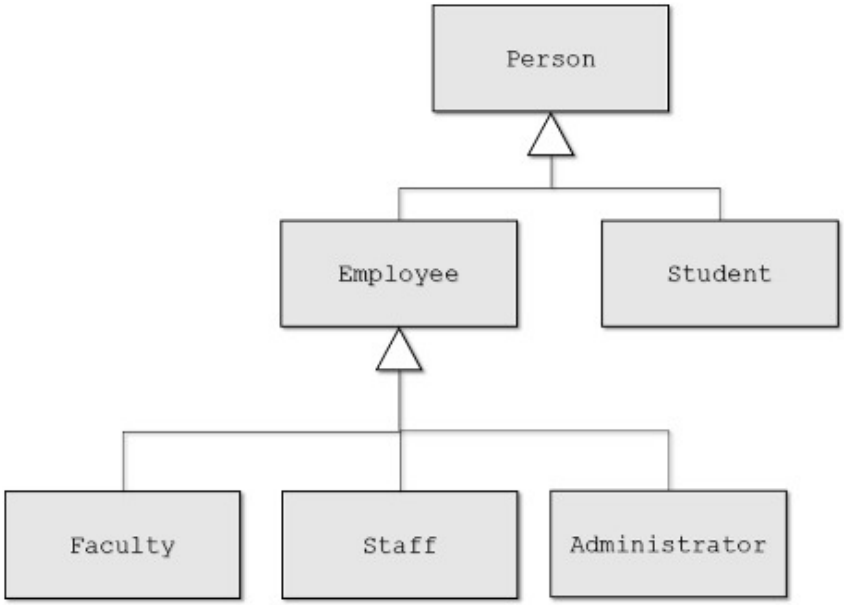

Figure 5: Inheritance

Héritage vs composition

Héritage public vs composition

- Les deux permettent la réutilisation des classes existantes
- Héritage public
  - Relation `Is-A`
    * L'employé « est-une personne »
    * Le compte courant « est-un compte »
    * Cercle `Est-une` Forme

+Composition + Relation `has-a` + La personne « possède un compte » + Le joueur a une attaque spéciale + Cercle `a un` emplacement

Héritage public vs composition

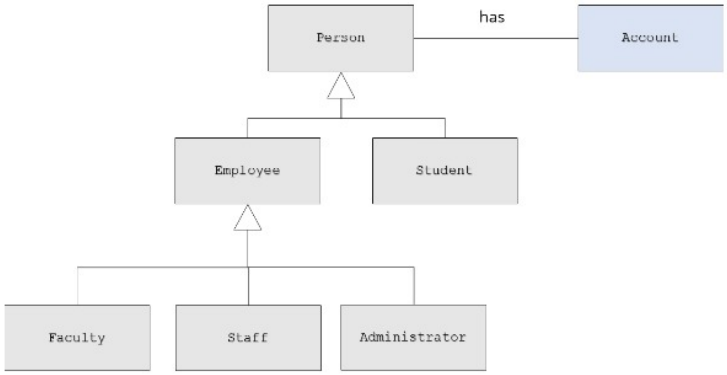

Figure 6: Inheritance

Composition

```
class Person {
private:
   std::string name;    // has-a name
   Account account; // has-a account
};
```

Figure 7: Inheritance

Dériver des classes à partir de classes existantes

- Syntaxe de dérivation `C++`
- Le spécificateur d'accès peut être : `public, privé ou protégé`
- Types d'héritage en `C++`
- `public`
  - Le plus courant
  - Établit une relation « est-une » entre les classes de base et
- `private` et `protected`
  - Établit une relation « la classe dérivée a une classe de base »
  - `Est mis en œuvre en termes de` relation
  - Différent de la composition

Syntaxe de dérivation `C++`

`C++` créant des objets

```
class Account {
  // Account class members . . .
};

class Savings_Account: public Account {
   // Savings_Account class members . . .
};

Savings_Account 'is-a' Account
```

Figure 8: Inheritance


```
class Account {
  // Account class members . . .
};

class Savings_Account: public Account {
   // Savings_Account class members . . .
};

Savings_Account 'is-a' Account
```


Figure 9: Inheritance

```
Savings_Account sav_account {};
Savings_Account *p_sav_account = new Savings_Account();

sav_account.deposit(1000.0);
p_sav_account->withdraw(200.0);

delete p_sav_account;
```

Figure 10: Inheritance

Membres protégés et accès aux cours

- Le modificateur de membre de classe protégé

```
class Base {

  protected:
    // protected Base class members . . .
};
```

Figure 11: Inheritance

- Accessible depuis la classe Base elle-même
- Accessible depuis les classes Dérivées de Base
- Non accessible par les objets de Base ou Dérivés
- Le modificateur de membre de classe protégé

```
class Base {
   public:
     int a;   // public Base class members . . .


   protected:
     int b;   // protected Base class members . . .


   private:
     int c;   // private Base class members . . .

};
```

Figure 12: Inheritance

- Accès avec héritage `public`
- Accès avec héritage `private`
- Accès avec héritage `protected`

Constructeurs et destructeurs

Constructeurs et initialisation de classe + Une classe dérivée hérite de sa classe Base

- La partie Base de la classe Derived DOIT être initialisée AVANT que la classe Derived soit initialisée
- Lorsqu'un objet dérivé est créé

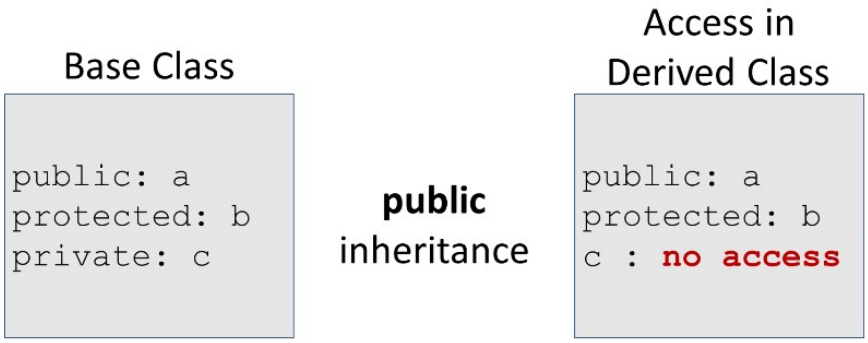

Figure 13: Inheritance

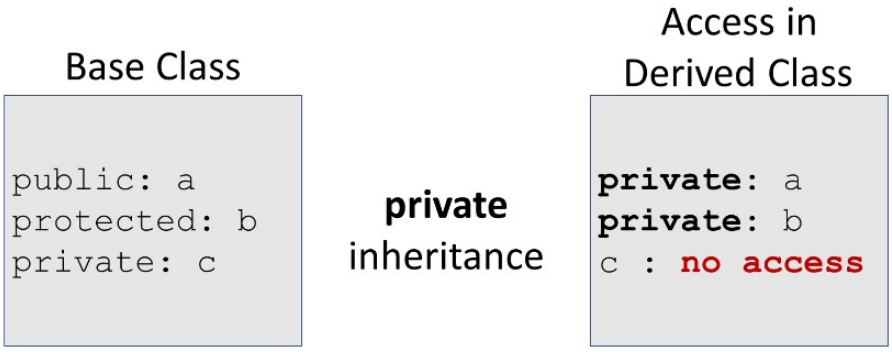

Figure 14: Inheritance

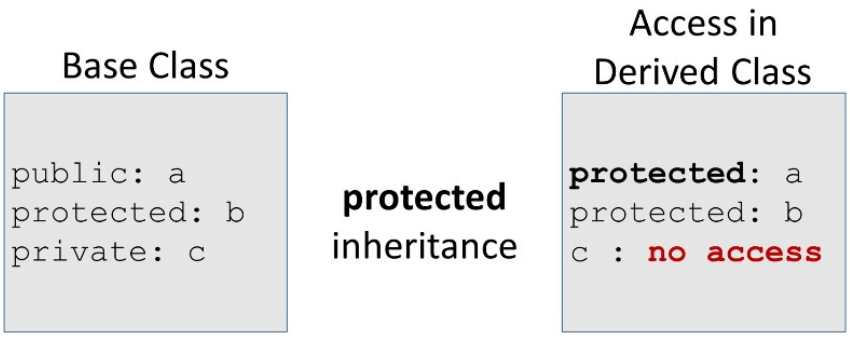

Figure 15: Inheritance

- Le constructeur de la classe de base s'exécute ensuite
- Le constructeur de classe dérivée s'exécute

```
class Base {
public:
   Base(){ cout << "Base constructor" << endl; }
};

class Derived : public Base {
public:
   Derived(){ cout << "Derived constructor " << endl; }
};
```

	Output
`Base base;`	`Base constructor`
`Derived derived;`	`Base constructor` `Derived constructor`

Destructeurs

- Les destructeurs de classes sont invoqués dans l'ordre inverse en tant que constructeurs
- La partie Derived de la classe Derived DOIT être détruite AVANT que le destructeur de classe Base ne soit invoqué
- Lorsqu'un objet dérivé est détruit
  - Le destructeur de classe dérivée s'exécute ensuite
  - Le destructeur de classe de base s'exécute
  - Chaque destructeur doit libérer les ressources allouées dans ses propres constructeurs

Destructeurs et initialisation de classe

Constructeurs et initialisation de classe

- Une classe dérivée n'hérite PAS
  - Constructeurs de classes de base
  - Destructeur de classe de base
  - Opérateurs d'affectation surchargés de classe de base
  - Fonctions d'ami de classe de base
- Cependant, les constructeurs de classes dérivées, les destructeurs et les opérateurs d'affectation surchargés peuvent invoquer les versions de la classe de base.

```
class Base {
public:
   Base(){ cout << "Base constructor" << endl; }
   ~Base(){ cout << "Base destructor" << endl; }
};

class Derived : public Base {
public:
   Derived(){ cout << "Derived constructor " << endl; }
   ~Derived(){ cout << "Derived destructor " << endl; }
};
```

Figure 16: Inheritance

	Output
`Base base;`	`Base constructor` `Base destructor`

`Derived derived;`	`Base constructor`
	`Derived constructor`
	`Derived destructor`
	`Base destructor`

Figure 17: Inheritance

- `C++11` permet l'héritage explicite des constructeurs de base `non spéciaux` avec
  - `en utilisant Base :: Base` ; n'importe où dans la déclaration de classe dérivée
  - Beaucoup de règles impliquées, il est souvent préférable de définir vous-même les constructeurs

Passer des arguments aux constructeurs de classes de base

Héritage

- Passer des arguments aux constructeurs de classes de base
  - La partie Base de la classe Derivated doit être initialisée en premier
  - Comment contrôler exactement quel constructeur de classe de base est utilisé lors de l'initialisation ?
  - Nous pouvons invoquer le constructeur de classe Base que nous souhaitons dans la liste d'initialisation de la classe Derivated.

```
class Base {
public:
   Base();
   Base(int);
   . . .
};

Derived::Derived(int x)
   : Base(x), {optional initializers for Derived} {
   // code
}
```

Figure 18: Inheritance

Constructeurs et Destructeurs + Constructeurs et initialisation de classe

```
class Base {
   int value;
public:
   Base(): value{0} {
      cout << "Base no-args constructor" << endl;
   }
   Base(int x) : value{x} {
      cout << "int Base constructor" << endl;
   }
};
```

```
class Derived : public Base {
   int doubled_value;
public:
   Derived(): Base{}, doubled_value{0} {
      cout << "Derived no-args constructor " << endl;
   }
   Derived(int x) : Base{x}, doubled_value {x*2} {
      cout << "int Derived constructor " << endl;
   }
};
```

	Output
`Base base;`	`Base no-args constructor`
`Base base{100};`	`int Base constructor`
`Derived derived;`	`Base no-args constructor` `Derived no-args constructor`
`Derived derived{100};`	`int Base constructor` `int Derived constructor`

Copier/Déplacer les constructeurs et l'opérateur = avec des classes dérivées

Copier le constructeur

- Peut invoquer explicitement le constructeur de copie de base
  - L'objet dérivé `other` sera `tranché`

```
Derived::Derived(const Derived &other)
   : Base(other), {Derived initialization list}
{
    // code
}
```

```
class Base {
   int value;
public:
   // Same constructors as previous example

   Base(const Base &other) :value{other.value} {
   cout << "Base copy constructor" << endl;
   }
};
```

```
class Derived : public Base {
   int doubled_value;
public:
   // Same constructors as previous example

   Derived(const Derived &other)
   : Base(other), doubled_value {other.doubled_value } {
         cout << "Derived copy constructor " << endl;
   }
};
```

Opérateur =

```
class Derived : public Base {
   int doubled_value;
public:
   // Same constructors as previous example
   Derived &operator=(const Derived &rhs) {
      if (this != &rhs) {
         Base::operator=(rhs);          // Assign Base part
         doubled_value = rhs.doubled_value; // Assign Derived part
      }
      return *this;
   }
};
```

Figure 19: Inheritance

Copier/Déplacer les constructeurs et l'opérateur =

- Souvent, vous n'avez pas besoin de fournir le vôtre
- Si vous ne les définissez pas dans Dérivé
  - Ensuite, le compilateur les créera automatiquement et appellera la version de la classe de base
- Si vous « FAIRE » fournissez des versions dérivées

  - Ensuite, « VOUS » devez invoquer vous-même les versions de base « explicitement »

- Soyez prudent avec les pointeurs bruts
  - Surtout si Base et Derived ont chacun des pointeurs bruts
  - Fournissez-leur une sémantique de copie approfondie

Redéfinition des méthodes de classe de base

Héritage

- Utilisation et redéfinition des méthodes de la classe Base
  - La classe dérivée peut invoquer directement les méthodes de la classe de base
  - La classe définie peut `remplacer` ou `redéfinir` les méthodes de la classe de base
  - Très puissant dans le contexte du polymorphisme

```
class Account {
public:
   void deposit(double amount) { balance += amount; }
};

class Savings_Account: public Account {
public:
   void deposit(double amount) { // Redefine Base class method
      amount += some_interest;
      Account::deposit(amount);  // invoke call Base class method
   }
};
```

Figure 20: Inheritance

Liaison statique des appels de méthode

- La liaison de la méthode à utiliser est effectuée au moment de la compilation
  - La liaison par défaut pour `C++` est statique
  - Les objets de classe dérivés utiliseront `Derived::deposit`
  - Mais, nous pouvons explicitement innover `Base::deposit` à partir de `Derived::deposit`
  - Ok, mais limité - une approche beaucoup plus puissante est la liaison dynamique que nous verrons dans la section suivante

Héritage multiple

- Une classe dérivée hérite de deux ou plusieurs classes de base en même temps

```
Base b;
b.deposit(1000.0);              // Base::deposit


Derived d;
d.deposit(1000.0);              // Derived::deposit


Base *ptr = new Derived();
ptr->deposit(1000.0);           // Base::deposit ????
```

Figure 21: Inheritance

- Les classes de base peuvent appartenir à des hiérarchies de classes non liées
- Un directeur de département
  - Faculté `Is-A` et
  - Administrateur `Is-A`

Syntaxe C++

- Au-delà du cadre de ce cours
- Quelques cas d'utilisation convaincants
- Facilement mal utilisé
- Peut être très complexe

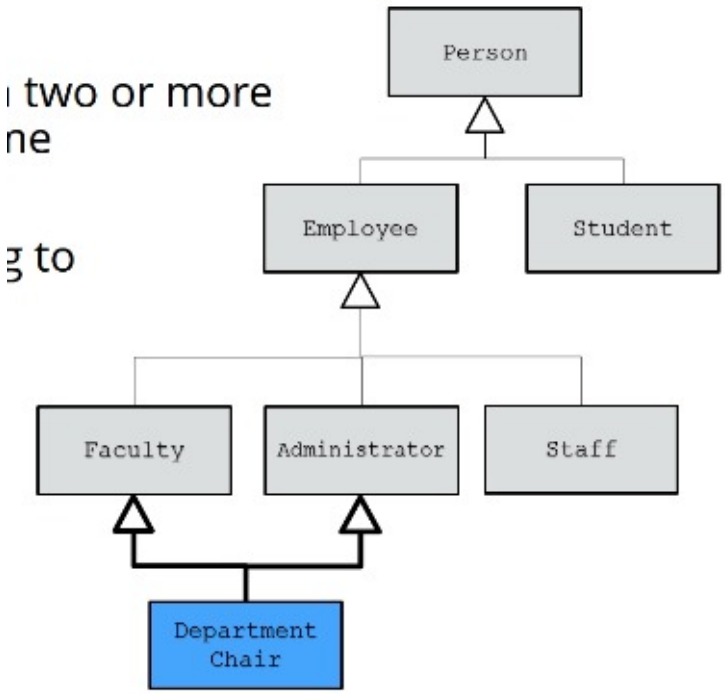

Figure 22: Inheritance

```
class Department_Chair:
      public Faculty, public Administrator {
   . . .
};
```

Figure 23: Inheritance